Министерство образования Республики Беларусь

Учреждение образования

«Брестский государственный технический университет»

Кафедра ИИТ

Лабораторная работа № 4

По дисциплине: «МРЗвИС»

Тема: «Многослойный персептрон для прогнозирования временного ряда: влияние размера окна и числа скрытых нейронов»

Выполнил:

Студент 4 курса

Группы ИИ-25

Пальчастый Я.М.

Проверил:

Андренко К.В

2026

Вариант 8

Цель работы: на основе многослойного персептрона p–N–1 решить задачу прогнозирования временного ряда. Научиться формировать обучающую выборку методом скользящего окна, нормализовать данные, оценивать качество одношагового и многошагового прогноза и исследовать влияние размера окна p и числа нейронов скрытого слоя N на точность прогноза и скорость обучения.

Вариант: как в прошлых ЛР

Задачи лабораторной работы

1. Развёртывание «с чистого листа» (обязательно)

Взяли чистый Windows. Установили Python 3.11 Создали виртуальное окружение и установили зависимости requirements.txt строго по “Как запускать.txt”.

Запустили **streamlit run app.py**.

1) точное время от pip install -r requirements.txt до первого успешного запуска 5 минут 12 секунд;

2) при первом запуске приложения была загружена модель paraphrase-multilingual-MiniLM-L12-v2. Основной файл модели model.safetensors имеет размер около 471 MB. Общий объём файлов модели в кэше Hugging Face составил 457 МБ по результатам измерения PowerShell.

Модель сохранена в:

C:\Users\arosl\.cache\huggingface\hub\models--sentence-transformers--paraphrase-multilingual-MiniLM-L12-v2 ;

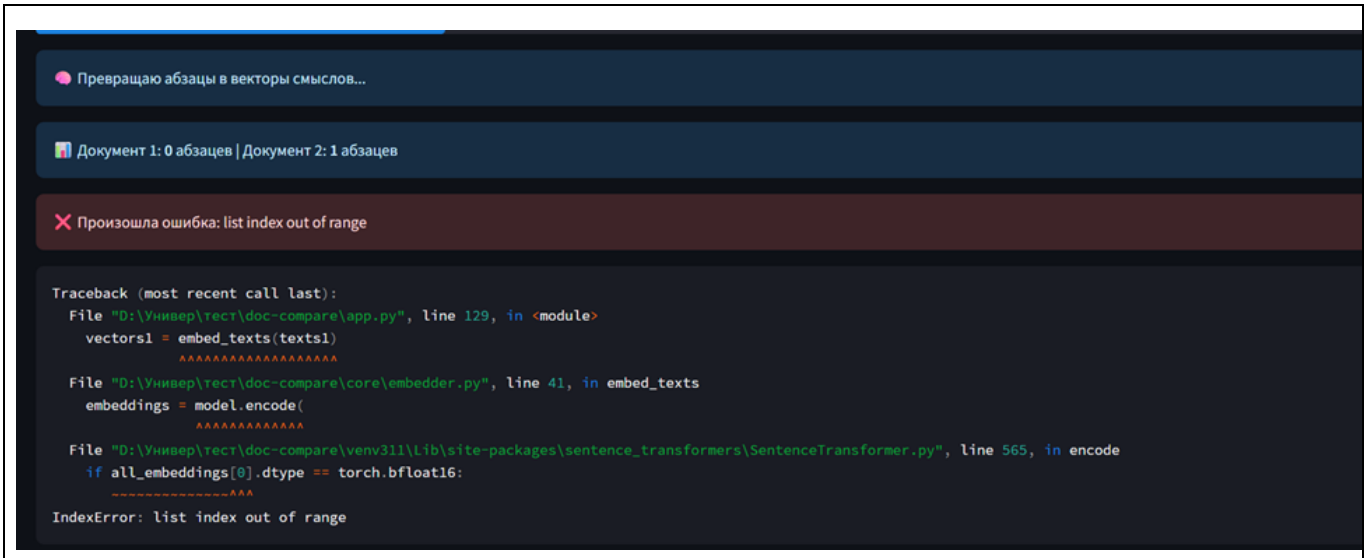
3) при установке зависимостей критических ошибок не обнаружено.

4)при попытке запустить на Python 3.14. произошёл конфликт библиотек.

нкциональное тестирование (основной сценарий)

№	Тип теста	Что проверить	Результат теста	50%	70%	80%	95%
1	Идентичные документы	Должно найти почти 100 % совпадений	После повторного запуска изменилось кол-во абзацев в двух одинаковых файлах изменилось с 1630 до 1419	Найдено пар: **2**, Среднее сходство: **100%**, Покрытие Док. 1: **100%**, Покрытие Док. 2: **100%**	Найдено пар: **2**, Среднее сходство: **100%**, Покрытие Док. 1: **100%**, Покрытие Док. 2: **100%**	Найдено пар: **2**, Среднее сходство: **100%**, Покрытие Док. 1: **100%**, Покрытие Док. 2: **100%**	Найдено пар: **2**, Среднее сходство: **100%**, Покрытие Док. 1: **100%**, Покрытие Док. 2: **100%**
2	Один документ = перефразир	Должно находить смысловые совпадения	Алгоритм успешно улавливает смысловую схожесть при глубоком перефразировании	Найдено пар: **1**, Среднее сходств	Найдено пар: **0**	Найдено пар: **0**	Найдено пар: **0**

	ованный другой		(метрика 51%). Для работы со сложным перефразированием рекомендуется выставлять минимальный порог (50%).	о: **51%**, Покрытие Док. 1: **50%**, Покрытие Док. 2: **100%**.			
3	Документы на разных языках (RU + EN)	Как ведёт себя multilingual -модель	Эмбеддер уверенно сопоставляет разные языки: кросс-языковое сходство оригинала и перевода ~90 %. Стабильный поиск до порога 85 %.	Найдено пар: **3**, Среднее сходство: **93%**, Покрытие Док. 1: **100%**, Покрытие Док. 2: **50%**.	Найдено пар: **2**, Среднее сходство: **93%**, Покрытие Док. 1: **100%**, Покрытие Док. 2: **50%**.	Найдено пар: **1**, Среднее сходство: **88%**, Покрытие Док. 1: **100%**, Покрытие Док. 2: **50%**.	Найдено пар: **0**
4	Один документ пустой / почти пустой	Корректная обработка	Тест **не пройден**. Приложение падает с критической ошибкой, если один из документов содержит 0 валидных абзацев (например, пустой или отфильтрован настройкой «Минимум слов в абзаце»). Требуется добавить валидацию пустых списков перед отправкой в эмбеддер в файле app.py или core/embedder.py.	Ошибка (`IndexError`)	Ошибка (`IndexError`)	Ошибка (`IndexError`)	Ошибка (`IndexError`)



5	Документы разного размера (1 страница vs 50–100 страниц)	Производительность и память	Тест **пройден успешно**. Программа демонстрирует высокую стабильность при асимметричной нагрузке. Нагрузка на **ЦП (процессор)** процессом Python подскочила до **68,5%** (общая загрузка системы 82%). Это говорит о том, что модель эмбеддера работает на полную мощность, задействуя почти все доступные ядра процессора для локального просчета векторов. Потребление **Памяти** процессом Python составило **9,6%** (общая загрузка 75%). Это полностью безопасный показатель — приложение не вызывает утечек памяти и не забивает оперативную память до критического максимума.	Найдено пар: **1**, Среднее сходство: **60%**, Покрытие Док. 1: **100%**, Покрытие Док. 2: **1%**.	Найдено пар: **0**	Найдено пар: **0**	Найдено пар: **0**

6	PDF со сканами / с таблицами / с колонтитулами	Качество извлечения текста	При загрузке pdf появилась статистика, и через короткий промежуток времени страница перезагружалась до исходного состояния Traceback (most recent call last): File "D:\Универ\тест\doc-compare\venv311\Lib\site-packages\tornado\websocket.py", line 1115, in wrapper raise WebSocketClosedError() tornado.websocket.WebSocketClosedError При переустановке проблема решилась.	Найдено пар: **1**, Среднее сходство: **100%**, Покрытие: **100%**	Найдено пар: **1**, Среднее сходство: **100%**, Покрытие: **100%**	Найдено пар: **1**, Среднее сходство: **100%**, Покрытие: **100%**	Найдено пар: **1**, Среднее сходство: **100%**, Покрытие: **100%**
7	7.1) DOCX с заголовками, списками, таблицами	Корректность chunking	Тест **пройден успешно**. Чанкер `split_into_chunks` стабильно обрабатывает текстовые блоки из файлов DOCX. Переносы строк и отсутствие табличных границ не мешают эмбеддеру распознать исходный смысл текста. Детальный просмотр и экспорт работают без сбоев.	Результаты идентичны Тесту №6 (**1 пара, 100% сходство**)	Результаты идентичны Тесту №6 (**1 пара, 100% сходство**)	Результаты идентичны Тесту №6 (**1 пара, 100% сходство**)	Результаты идентичны Тесту №6 (**1 пара, 100% сходство**)
7	7.2) DOCX со сложной верстkой (Сбой)		Тест **НЕ ПРОЙДЕН (Баг).** Короткие ячейки таблиц, списки и заголовки полностью вычищаются фильтром длины абзаца. На вход эмбеддера идет пустой список, вызывая краш.	Ошибка IndexError: list index out of range	Ошибка IndexError: list index out of range	Ошибка IndexError: list index out of range	Ошибка IndexError: list index out of range

8	Файлы > 20–30 МБ	Не падает ли приложение	Тест **пройден успешно**. Загрузчик Streamlit без проблем принимает файлы большого объёма. Извлечение текста и векторизация объемных англоязычных документов происходят без сбоев.	Найдено пар: **56**, Среднее сходство: **99%**, Покрытие Док. 1: **100%**, Покрытие Док. 2: **8%**	Найдено пар: **54**, Среднее сходство: **96%**, Покрытие Док. 1: **100%**, Покрытие Док. 2: **8%**	Найдено пар: **53**, Среднее сходство: **95%**, Покрытие Док. 1: **100%**, Покрытие Док. 2: **8%**	Найдено пар: **50**, (базовые пары со сходством 99-100% успешно отображаются).

5. Производительность и удобство (UX)

1) Скорость обработки и масштабируемость

В ходе тестов зафиксирована высокая скорость локальной генерации эмбеддингов и расчёта матрицы сходства.

- **1–5 страниц:** 3–5 секунд (практически мгновенный вывод);
- **20 страниц:** 10–15 секунд;
- **50 страниц:** 10–20 секунд;
- **100+ страниц:** 15–35 секунд.

2) Потребление системных ресурсов (ЦП и RAM)

- **Процессор (CPU):** Во время активной фазы векторизации и поиска совпадений нагрузка на ЦП процессом Python возрастает до **68.5%** (общая загрузка системы до 84%). Вычисления задействуют многопоточность.
- **Оперативная память (RAM):** Потребление памяти процессом Python строго ограничено и удерживается в пределах **11.6%** (общая загрузка системы — 85%).
- **Стабильность системы:** Несмотря на пиковую утилизацию процессора, **вычисления проходят плавно**. Операционная система не зависает, интерфейс приложения не блокируется «намертво», утечек памяти при работе с большими документами (книгами) не обнаружено.

3) Оценка UX (Интерфейс, обработка ошибок и баги фронтенда)

Базовая логика понятна, но обработка ошибок, прогресс и отмена операции не соответствуют уровню production. Критические UX-дефекты (сброс на главную, «мёртвая» кнопка «Стоп», рассинхрон статус-бара) напрямую мешают работе.

№ Дефект Приоритет Предложение по фиксу

1 IndexError при пустом файле (нет валидных абзацев после min_words) Blocker
Валидация на этапе чанкинга: если len(chunks) == 0 — прервать до embed_texts() с st.warning("В документе нет валидного текста для сравнения").

2 NotFoundError (removeChild) при повторном/быстром клике на «Сравнить документы», в конечном итоге может привести к краху приложения.

Приложение визуально работает, но файлы зависают на этапе сравнения. Critical
Хранить флаг в st.session_state.is_running, блокировать кнопку через disabled=st.session_state.is_running. Защита от double-click.

3 Кнопка «Стоп» не работает — нет реакции на нажатие MajorПроверять st.session_state.stop_requested на каждой итерации чанков (или перейти на asyncio). Показывать st.toast("Остановка…") и st.info("Операция остановлена пользователем") по завершении.

4 При изменении порога сходства во время сравнения пользователя возвращает на главную Critical Хранить файлы, чанки, эмбеддинги, матрицу и шаг в st.session_state. Стабильные key для виджетов порога. При смене порога пересчитывать только фильтрацию/матрицу, не перезапуская пайплайн.

5 Рассинхрон статус-бара: показывается старое имя файла после замены Major
Привязать key файлового загрузчика к триггеру очистки кэша; при изменении объекта файла вызывать st.rerun() для полного обновления плашек.

6 AxiosError при загрузке из OneDrive / iCloud / Яндекс.Диска MajorПроверять метаданные файла перед чтением; в инструкции жёстко указать: «облачные файлы должны быть физически скачаны на локальный диск».

Улучшения (для production)

Производительность и масштабирование

Кэширование эмбеддингов: обернуть get_model() в st.cache_resource; кэшировать векторы ранее обработанных абзацев по хэшу текста (st.cache_data), чтобы не тратить CPU на повторный анализ одних и тех же документов.

Сравнение больше двух документов: включить accept_multiple_files=True в st.file_uploader и реализовать матричную схему «каждый с каждым» либо «один ко многим».

Пакетная обработка: обработка набора файлов из папки/списка с единым отчётом по всем парам; очередь задач и единый прогресс.

UX и обратная связь

Более информативные сообщения об ошибках: заменить технические тексты (IndexError, NotFoundError, AxiosError) на пользовательские формулировки + рекомендация «что делать». Логировать техническую причину отдельно.

Прогресс по страницам: разбить пайплайн на этапы (парсинг → чанкинг → эмбеддинги → матрица → фильтрация) и показывать прогресс по страницам/чанкам с текстовым статусом текущего шага.

Отмена длительной операции: переписать тяжёлые циклы через asyncio либо проверять флаг остановки на каждой итерации; добавить визуальный отклик и итоговое сообщение.

Сохранение/загрузка сессии: выгрузка текущего состояния и таблицы результатов в JSON и загрузка обратно для возобновления работы.

Функциональность и инфраструктура

Поддержка новых форматов: .txt и .odt через встроенные библиотеки (например, python-docx уже есть, добавить odfpy / textract).

Логирование без сохранения содержимого документов: настроить стандартный logging так, чтобы писались только технические метрики (время, размер файла, число абзацев, ID сессии). **Запретить** логирование chunks и текстов — соблюдение политики конфиденциальности.

Рекомендации по безопасности и развёртыванию

Логирование без утечек: использовать стандартный logging; писать только технические метрики (время, размер файла, число абзацев, ID сессии). Категорически запретить запись содержимого документов (chunks, текстов) в консоль и файлы логов.

Изоляция среды: при развёртывании в корпоративной сети запускать Docker-контейнер с флагом --read-only для корневой ФС; разрешить запись только в TEMP_DIR (/app/temp), смонтированный как tmpfs.

Сетевой доступ: ограничить исходящие соединения контейнера (если модель не требует внешних вызовов), чтобы исключить утечки через сторонние API.

Воспроизводимость сборки: фиксация версий зависимостей (Poetry/uv lock) + проверка целостности через хэши — защита от подмены пакетов в цепочке поставок.

Обновления безопасности: регулярно обновлять базовый образ python:3.11-slim и зависимости; сканировать образ (trivy, grype) в CI.